

```
[root@deepu waf]# ls
data.o main.o show.o
[root@deepu waf]# gcc -c main.c
[root@deepu waf]# gcc -c data.o
[root@deepu waf]# gcc -c show.o
[root@deepu waf]# gcc main.o data.o show.o -o hello
[root@deepu waf]# ls
data.o data.o hello main.o main.o show.o show.o
[root@deepu waf]# ./hello

Let us learn about Waf

Waf is a build automation tool
```

Figure 1: Manual compilation output

also makes sure that the linker is not called into action. The first three commands create three object files called *main.o*, *data.o* and *show.o*. The fourth command uses the object files *main.o*, *data.o* and *show.o* to produce an executable file called *hello* with the option *-o*. This executable *hello* is executed with the command *./hello* and the output is shown in Figure 2.

A simple Waf script

What if the program contains 10, 100 or 1000 files? Well, we need 11, 101 or 1001 commands, respectively, to generate an executable file. This is alarming and almost as difficult as writing the program itself. But here comes Waf to our rescue. The same effect as one thousand and one command line instructions can be achieved with the help of a single Waf script. The Waf executable will search for a script called *wscript* containing all the rules to build a particular project in the current working directory of the project from which the Waf executable is called. The code below shows the *wscript* to compile and build the sample project.

```
top = '.'
out = '.'
def options(opt):
    opt.load('compiler_c')
def configure(conf):
    conf.load('compiler_c')
def build(bld):
    bld.program(source='main.c', target='hello',
    use='data show')
    bld.objects(source='data.c', target='data')
    bld.objects(source='show.c', target='show')
def test(tst):
    print("\nThis is a waf test function\n\n")
```

Always remember to save the Waf script files with the name *wscript*. In order to build our sample project, open a terminal and set the directory containing the sample C programs and the Waf script *wscript* as the current working directory. If you have manually created the executable file *hello*, make sure that you have deleted the

```
[root@deepu waf]# waf configure
Setting top to : /root/Desktop/waf
Setting out to : /root/Desktop/waf
Checking for gcc (cc compiler) : /usr/lib64/gcc/gcc
'configure' finished successfully (0.135s)
[root@deepu waf]# waf
Waf: Entering directory '/root/Desktop/waf'
Building from the build directory, forcing --targets=
[1/4] c: main.o -> main.o:1.o
[2/4] c: data.c -> data.o:2.o
[3/4] c: show.c -> show.o:3.o
[4/4] cprogram: main.o:1.o data.o:2.o show.o:3.o -> hello
Waf: Leaving directory '/root/Desktop/waf'
'build' finished successfully (0.092s)
[root@deepu waf]# ls
c-ache config-logs data.o data.o:2.o hello main.o main.o:1.o show.o show.o:3.o wscript
Let us learn about Waf

Waf is a build automation tool
```

Figure 2: Output of *wscript* execution

executable as well as the object files. The sample project can be built by executing the following commands.

```
waf configure
waf
```

If you haven't renamed the file *waf-1.9.6* as *waf*, then you need to execute the command *waf-1.9.6* on the terminal instead of *waf*. Figure 2 shows the output of these command executions.

Figure 2 also shows the associated files generated by Waf other than the executable *hello*. So it is better to store the output data into a separate directory other than the current working directory. For example, the line of code *out = './dest'* instead of the line of code *out = '.'* will store the executable *hello* and other associated directories into a directory called *dest* inside the current working directory. And now, if you run the executable *hello* in the current working directory or the directory *dest* with the command *./hello* you will get the same output shown in Figure 2.

The Waf script demystified

Now, it is time for us to understand the working of the Waf script file *wscript*. The line *'top = .'* defines the top directory of the project. This is almost always the directory containing the top level *wscript* of your project. Remember that *'.'* denotes the current working directory in Linux and this is where Waf initially searches for source files, unless you are recursively calling other Waf scripts from one of the parent directories or sub-directories (a topic that we are going to discuss later). The line *out = '.'* defines the name of the output directory to which the executable file of the project and other dependent files are to be saved. As mentioned earlier, in our case, the executable file will be saved to the current working directory.

If you are familiar with the programming language Python, you will see that the remaining lines of code in the file *wscript* are nothing but the definition of four Python functions—*options*, *configure*, *build* and *test*. Even if you don't know Python, there's no need to worry because you don't need Python to use Waf. The most